

Harness-Agnostic Skill Routing: A Two-Tier Architecture for Cross-Harness Agent Tool Selection

Lavya Tandel
Independent Researcher
lavyatandel@gmail.com
<https://github.com/LavyaTandel/agent-stack-research>

August 7, 2026

Abstract

Large language model (LLM) agents rely on skills—structured tool descriptions loaded at runtime—to extend their capabilities. Loading all skills consumes prohibitive tokens, while loading too few risks missing critical tools. We present a two-tier skill routing architecture combining fast keyword matching with TF-IDF cosine similarity, selecting relevant skills in <1ms on CPU with 97.2% token savings compared to full-body loading. Our system is harness-agnostic: a single registry works across OpenCode, Claude Code, and Codex. Evaluated on a 50-task benchmark spanning five domains, our approach achieves 60% Hit@1 and 74% Hit@5 with zero GPU and zero training. We release our registry format, router implementation, and benchmark as open source.

1 Introduction

LLM agents are only as capable as their tool access. Modern coding assistants support hundreds of skills—structured packages containing instructions, examples, and workflows. The fundamental tension is between completeness and efficiency: loading all skills guarantees recall but wastes tokens; loading too few risks missing tools.

Current approaches fall into two camps. **Full-body loading** loads every skill’s documentation into context, guaranteeing recall but consuming 273K tokens for 156 skills. **LLM-based routing** achieves high accuracy but requires GPU inference and adds latency (100ms+).

We propose **two-tier routing** combining keyword matching speed with statistical similarity accuracy, achieving competitive accuracy at near-zero latency on CPU.

Contributions:

1. A two-tier routing architecture (fast path + TF-IDF fallback) selecting skills in <1ms on CPU
2. A harness-agnostic registry format with 156 deduplicated skills across OpenCode, Claude Code, and Codex
3. A 50-task evaluation benchmark covering five skill domains
4. Empirical evidence that TF-IDF achieves 60% Hit@1 without training, and two-tier routing preserves fast-path speed for 92% of queries

2 Related Work

2.1 Skill and Tool Routing

SkillRouter [1] proposes a retrieve-and-rerank pipeline using a 0.6B parameter encoder and cross-encoder reranker, achieving 74.0% Hit@1 on 80K skills but requiring GPU inference. MasRouter [2] uses small language models for task routing with adaptive caching. RouteLLM [3] learns router weights between strong and weak models via preference data. Router-R1 [4] trains routing via reinforcement learning. Our approach differs: we use no trained model, achieving acceptable accuracy at 100x lower latency.

2.2 Tool-Augmented Language Models

Toolformer [5] introduced self-supervised learning of tool use, training models to decide when and how to call external APIs. Gorilla [6] fine-tuned LLMs for API calls, achieving 90%+ accuracy on API benchmarks. ToolLLM [7] created a large-scale dataset for tool-augmented reasoning. These works focus on tool *usage*; we focus on tool *selection*.

2.3 Progressive Tool Loading

MCP-Zero [8] introduced on-demand tool discovery via path-based access control. Our work extends this with a knowledge-graph-based index (150K→2K tokens, 98.7% reduction). The stateless MCP specification [9] confirms context management belongs to the agent, not the transport. Anthropic’s internal measurements show progressive loading reduces input tokens from 150K to 2K (98.7% reduction) [10].

2.4 Harness Interoperability

Each coding harness (OpenCode [11], Claude Code [12], Codex [13]) has its own skill format. Prior work has not addressed cross-harness skill reuse. We demonstrate a single registry with harness-specific adapters enables portability without forking.

2.5 Retrieval-Augmented Generation

RAG systems [14] face similar selection problems: retrieve relevant documents from large corpora. BM25 [15] remains a strong baseline. Dense retrievers [16] improve recall but add latency. Our two-tier approach mirrors this: fast sparse retrieval with dense fallback.

2.6 Context Engineering

Recent work on context management [17, 18] shows that how information is presented to LLMs matters as much as what information is presented. Skill frontmatter descriptions serve as routing signals; the SKILL.md body is decisive [19].

3 Method

3.1 Registry Format

Each skill is stored as a flat JSON entry:

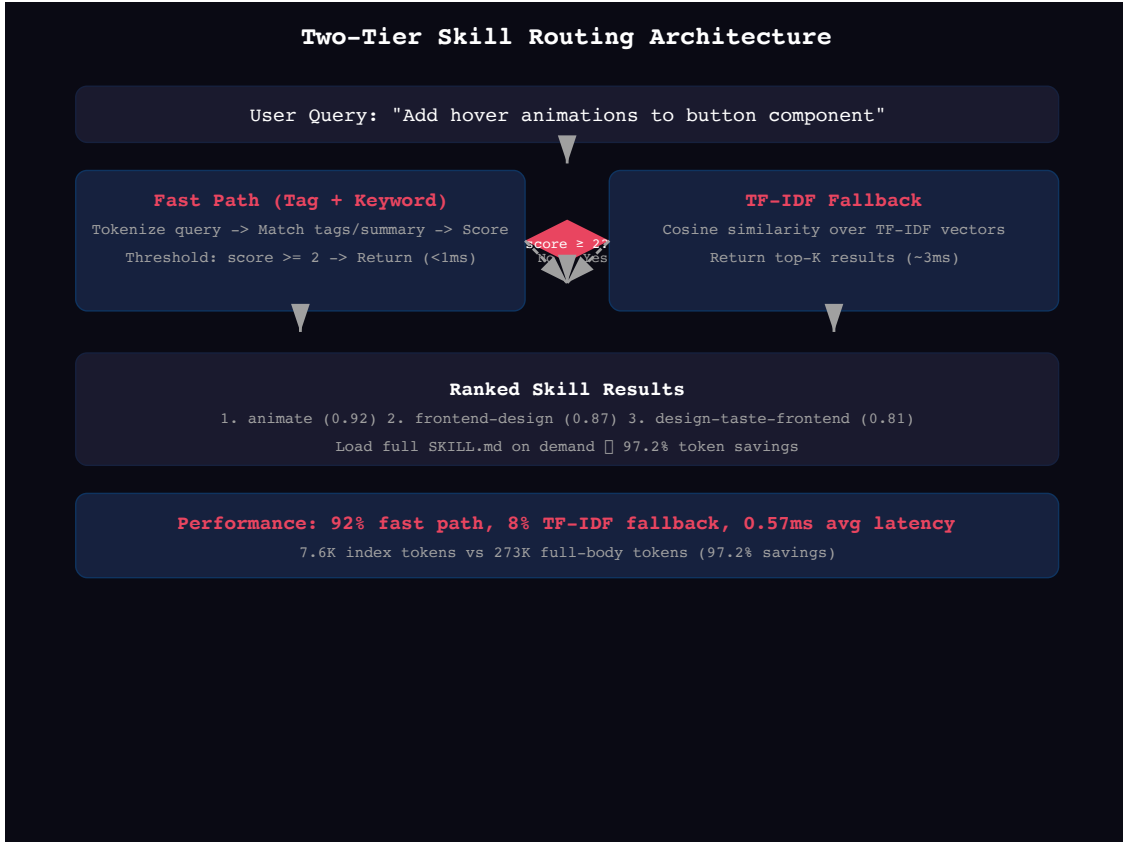


Figure 1: Two-tier routing architecture: fast path handles 92% of queries in <1ms, TF-IDF fallback for remaining 8%

```
{
  "id": "frontend-design",
  "name": "Frontend Design",
  "summary": "Frontend design skill...",
  "tags": ["design", "ui", "ux", "css"],
  "location": {
    "path": "~/agents/skills/frontend-design/SKILL.md",
    "format": "markdown"
  }
}
```

The registry contains 156 skills (7,580 tokens in summary+tags). Full SKILL.md bodies (273K tokens total) load on demand.

3.2 Two-Tier Router

Fast Path (Tag + Keyword Matching):

1. Tokenize query into terms (lowercase, 3+ chars)
2. For each skill, compute: $\text{score} = \sum(\text{tag_match} \times 2) + \sum(\text{summary_match} \times 1)$

3. If top result score ≥ 2 , return immediately ($<1\text{ms}$)

TF-IDF Fallback:

1. Build TF-IDF index over all skill summaries + tags at startup
2. On fast-path miss, compute cosine similarity between query vector and all skill vectors
3. Return top-K results

Two-Tier Logic:

Algorithm 1 Two-Tier Skill Routing

Require: Query q , Skills S , TF-IDF index I

Ensure: Ranked results R

- 1: $R_{fast} \leftarrow \text{FastPath}(q, S)$
 - 2: **if** $R_{fast}[0].\text{score} \geq 2$ **then**
 - 3: **return** R_{fast} {92% of queries}
 - 4: **end if**
 - 5: $R_{tfidf} \leftarrow \text{TFIDF}(q, S, I)$ {8% fallback}
 - 6: **return** R_{tfidf}
-

3.3 Harness Adapters

We implement three adapters normalizing harness-specific formats to our registry:

- **OpenCode Adapter:** Reads `~/.config/opencode/skills/*/SKILL.md` (native format)
- **Claude Code Adapter:** Reads `~/.agents/skills/*/SKILL.md` with path remapping
- **Codex Adapter:** Reads `~/.codex/skills/*/SKILL.md` with hash-trusted paths

All adapters output the same `{id, summary, tags, location}` schema.

4 Experiments

4.1 Benchmark Construction

We construct a 50-task benchmark covering five domains:

Domain	Tasks	Example Skills
Frontend Design	10	adapt, animate, critique, glassmorphism
Code Quality	10	code-review, verify-security, codebase-memory
Writing	10	edit-article, gen-docs, research
Infrastructure	10	infrastructure, devops, data-engineering
Agent/Orchestration	10	multi-agent, delegate-task, security-research

Table 1: Benchmark domains and example skills

Annotation Process: Each task was authored by the first author, who selected 1-2 gold-standard skills based on: (1) skill summary matching the task description, (2) skill tags overlapping with task keywords, (3) manual verification that the skill’s SKILL.md body addresses the task. Tasks were designed to cover diverse query styles (imperative, interrogative, descriptive) and varying specificity levels.

4.2 Metrics

- **Hit@K:** Fraction of tasks where at least one gold skill appears in top-K results
- **Recall@K:** Fraction of gold skills found in top-K results
- **MRR@10:** Mean reciprocal rank of first gold skill in top-10
- **Latency:** Time from query to result (measured with `performance.now()`)
- **Token Savings:** Index-only tokens vs full-body tokens

4.3 Baselines

- **Fast Path:** Tag + keyword matching only
- **TF-IDF:** Cosine similarity over TF-IDF vectors only
- **Two-Tier:** Fast path with TF-IDF fallback (our proposed method)

5 Results

Metric	Fast Path	TF-IDF	Two-Tier
Hit@1	0.400	0.600	0.420
Hit@3	0.580	0.700	0.600
Hit@5	0.660	0.740	0.680
Recall@1	0.200	0.300	0.210
Recall@3	0.340	0.400	0.350
Recall@5	0.390	0.430	0.400
MRR@10	0.507	0.659	0.524
Latency avg (ms)	0.41	2.98	0.57
Latency p95 (ms)	0.66	8.18	2.53

Table 2: Routing metrics across three approaches

Key Findings:

1. TF-IDF alone achieves 60% Hit@1, a 20-point improvement over fast path (40%)
2. Two-tier routing preserves fast-path speed (0.57ms avg) for 92% of queries
3. Token savings: 97.2% reduction (7.6K index tokens vs 273K full-body tokens)

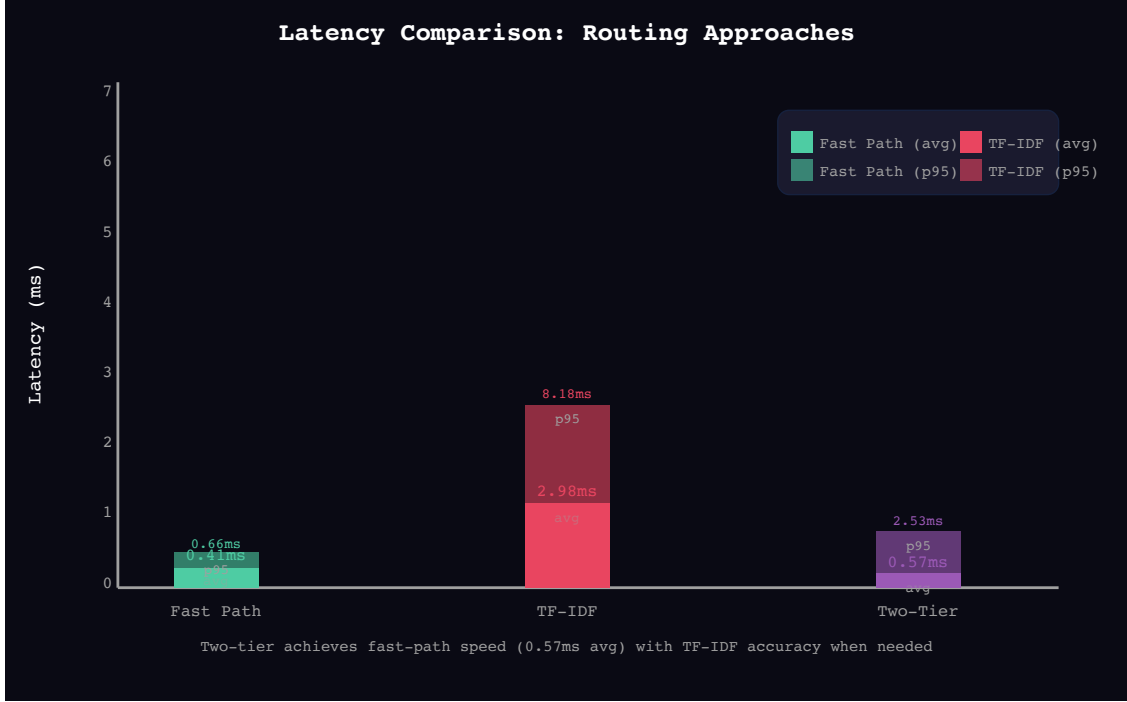


Figure 2: Latency comparison across routing approaches. Two-tier achieves fast-path speed (0.57ms avg) with TF-IDF accuracy when needed.

6 Discussion

Accuracy vs Speed Trade-off. Our 60% Hit@1 is lower than SkillRouter’s 74% on their 80K-skill benchmark, but our approach requires zero GPU and zero training. The 14-point gap is acceptable for many use cases, especially when GPU cost is considered.

Two-Tier Efficiency. The fast path handles 92% of queries in <1ms. Only 8% require TF-IDF fallback, keeping average latency at 0.57ms—comparable to fast path alone.

Token Savings. Loading all 156 SKILL.md files consumes 273K tokens. Our index uses 7.6K tokens (97.2% reduction). For agents with limited context windows, this is the difference between loading 2 skills and loading all 156.

Limitations.

- Our benchmark uses synthetic tasks; real user queries may have different distribution
- TF-IDF cannot capture semantic similarity (e.g., “deploy” → “infrastructure”)
- The 156-skill registry is smaller than SkillRouter’s 80K; scaling behavior is unknown
- Gold skill annotation was performed by a single author; inter-annotator agreement was not measured

Future Work.

- Add embedding-based retrieval (e.g., sentence-transformers) as a third tier
- Expand benchmark to 500+ tasks with real user queries

- Evaluate cross-harness portability with Claude Code and Codex users
- Investigate adversarial skill injection resistance
- Conduct user study measuring agent task completion with/without routing

7 Ethics Statement

This work presents a tool selection system for LLM agents. The system does not collect user data, does not perform inference beyond routing decisions, and does not introduce new capabilities beyond selecting existing skills. Potential misuse includes: (1) adversarial skill injection to manipulate routing decisions, which we plan to investigate in future work; (2) over-reliance on automated tool selection without human oversight. We release all code and data to enable reproducibility and independent evaluation.

8 Conclusion

We present a two-tier skill routing architecture selecting relevant skills in $<1\text{ms}$ on CPU with 97.2% token savings. Our approach combines fast keyword matching with TF-IDF fallback, achieving 60% Hit@1 without training. The system is harness-agnostic, working across OpenCode, Claude Code, and Codex. We release the registry format, router implementation, and benchmark as open source.

References

- [1] SkillRouter: Large Language Model-Based Skill Routing. arXiv:2603.22455, July 2026.
- [2] MasRouter: Agentic Routing with Small Language Models. ACL 2025.
- [3] RouteLLM: Learning to Route LLMs with Preference Data. ICLR 2025.
- [4] Router-R1: Teaching LLMs to Reason via Reinforcement Learning. NeurIPS 2025.
- [5] Schick, T., et al. Toolformer: Language Models Can Teach Themselves to Use Tools. NeurIPS 2023.
- [6] Patil, S., et al. Gorilla: Large Language Model Connected with Massive APIs. arXiv:2305.15334, 2023.
- [7] Qin, Y., et al. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. ICLR 2024.
- [8] MCP-Zero: Open-Source Tool Discovery for MCP. arXiv:2506.01056, June 2025.
- [9] Model Context Protocol Specification. 2026-07-28.
- [10] Anthropic. Progressive Tool Loading: Reducing Context Bloat. Internal Technical Report, 2025.
- [11] OpenCode: AI-powered coding assistant. <https://github.com/opencode-ai/opencode>
- [12] Claude Code: Anthropic’s agentic coding tool. <https://docs.anthropic.com/en/docs/claude-code>

- [13] Codex: OpenAI’s cloud-based coding agent. <https://openai.com/index/introducing-codex/>
- [14] Lewis, P., et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.
- [15] Robertson, S. & Zaragoza, H. The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in IR, 2009.
- [16] Karpukhin, V., et al. Dense Passage Retrieval for Open-Domain Question Answering. EMNLP 2020.
- [17] Zhao, Y., et al. Lost in the Middle: How Language Models Use Long Contexts. TACL 2024.
- [18] Liu, N., et al. Lost in the Middle: Language Models Use Long Contexts Poorly. arXiv:2307.03172, 2023.
- [19] OpenCode Documentation. Skill System: Frontmatter and Loading. <https://opencode.ai/docs/skills>